C++ CODE & OUTPUT REPORT

FILE: 01_io_library.cpp

CODE:

```
#include<iostream>
int main(){
    std::cout<<"Enter 2 nos: "<<std::endl;
    int a,b;
    std::cin>>a>>b;
    std::cout<<"Sum is: "<<a+b<<std::endl;

    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\01_io_library.exe' timed out after 10 seconds
```

FILE: 02_switch.cpp

CODE:

```
#include<iostream>
#include<string>
int main(){
    std::string input;
    int aCnt=0, eCnt=0, iCnt=0, oCnt=0, uCnt=0;

    std::cout << "Enter a string: ";
    std::getline(std::cin, input);

    for(char ch : input){
        switch(ch){
            case 'a':
            case 'A':
                ++aCnt;
                break;
            case 'e':
            case 'E':
                ++eCnt;
                break;
            case 'i':
            case 'I':
                ++iCnt;
                break;
            case 'o':
            case 'O':
                ++oCnt;
                break;
            case 'u':
            case 'U':
                ++uCnt;
                break;
        }
    }

    std::cout << "Number of a's: " << aCnt << std::endl;
    std::cout << "Number of e's: " << eCnt << std::endl;
    std::cout << "Number of i's: " << iCnt << std::endl;
    std::cout << "Number of o's: " << oCnt << std::endl;
    std::cout << "Number of u's: " << uCnt << std::endl;
    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\02_switch.exe' timed out after 10 seconds
```

FILE: 03_do_while.cpp

CODE:

```
#include<iostream>
int main(){
    int i=0;
    do{
        std::cout<<i<<"\n";
        ++i;
    }while(i<5);
    return 0;
}
```

OUTPUT:

```
0
1
2
3
4
```

FILE: 04_goto.cpp

CODE:

```
#include<iostream>
using namespace std;
int main(){
    int i=0;
    start:
        cout<<i<<"\n";
        ++i;
        if(i<5)
            goto start;
    return 0;
}
```

OUTPUT:

```
0
1
2
3
4
```

FILE: 05_comma_operator.cpp

CODE:

```
#include<iostream>
using namespace std;
int main(){
    int a=10, b=20, c=30;
    int result = (a, b, c); // The comma operator evaluates all expressions and returns the last one
    cout << "Result of (a, b, c): " << result << endl; // Output will be 30
    return 0;
}
```

OUTPUT:

```
Result of (a, b, c): 30
```

FILE: 06_namespace.cpp

CODE:

```
#include<iostream>
using std::cout;
using std::endl;
using std::cin;

int main(){
    cout<<"Enter 2 nos: "<<endl;
    int a,b;
    cin>>a>>b;
    cout<<"Sum is: "<<a+b<<endl;

    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\06_namespace.exe' timed out after 10 seconds
```

FILE: 07_typedef.cpp

CODE:

```
#include<iostream>
using namespace std;

int main(){
    typedef int MyInt; // Define MyInt as an alias for int
    MyInt a = 10; // Now we can use MyInt instead of int
    cout << "Value of a: " << a << endl; // Output will be 10
    return 0;
}
```

OUTPUT:

```
Value of a: 10
```

FILE: 08_typedef_struct.cpp

CODE:

```
#include<iostream>
using namespace std;

typedef struct {
    int id;
    string name;
} Student;
int main(){
    Student s1;
    s1.id = 1;
    s1.name = "Alice";
    cout << "Student ID: " << s1.id << ", Name: " << s1.name << endl;
    return 0;
}
```

OUTPUT:

```
Student ID: 1, Name: Alice
```

FILE: 09_enumerations.cpp

CODE:

```
#include<iostream>
using namespace std;
enum Color { RED, GREEN=2, BLUE }; // Define an enumeration for colors
enum suit { HEARTS, DIAMONDS=10, CLUBS, SPADES }; // Define an enumeration for card suits
int main(){
    Color c = RED;
    suit s = DIAMONDS;
    cout << "Color: " << c << endl;
    cout << "Suit: " << s << endl;

    cout<<"The value of enum color :"<<RED<<","<<GREEN<<","<<BLUE<<endl;
    cout<<"The value of enum suit :"<<HEARTS<<","<<DIAMONDS<<","<<CLUBS<<","<<SPADES<<endl;
    return 0;
}
```

OUTPUT:

```
Color: 0
Suit: 10
The value of enum color :0,2,3
The value of enum suit :0,10,11,12
```

FILE: 10_string.cpp

CODE:

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string s;
    cout<<"Enter a string: ";
    cin>>s;
    cout<<"You entered: "<<s<<endl;
    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\10_string.exe' timed out after 10 seconds
```

FILE: 11_full_string.cpp

CODE:

```
#include<iostream>
using namespace std;
int main(){
    string s;
    cout<<"Enter a string: ";
    while(cin>>s){ // Read input until EOF
        cout<<s<<" "; // Print the entered string followed by a space
    }
    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\11_full_string.exe' timed out after 10 seconds
```

FILE: 12_string_better.cpp

CODE:

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string s;
    cout<<"Enter a string: ";
    getline(cin, s); // Read a full line of input into the string variable 's'
    cout<<"You entered: "<<s<<endl; // Output the entered string
    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\12_string_better.exe' timed out after 10 seconds
```

FILE: 13_string_operations.cpp

CODE:

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string first,second,third;
    cout<<"Enter 1st string: ";
    getline(cin, first);
    cout<<"Enter 2nd string: ";
    getline(cin, second);
    third=first +" " + second;
    cout<<"Concatenated string: "<<third<<endl;

    cout<<"You entered:"<<endl;
    cout<<"1st: "<<first<<endl;
    cout<<"2nd: "<<second<<endl;
    cout<<"3rd: "<<third<<endl;

    cout<<"Length of 1st string: "<<first.size()<<endl; // Get the length of the first string
    cout<<"Length of 2nd string: "<<second.length()<<endl; // Get the length of the second string
    cout<<"Length of 3rd string: "<<third.size()<<endl; // Get the length of the third string

    cout<<"Is 1st string empty? "<<first.empty()<<endl; // Check if the first string is empty
    cout<<"Is 2nd string empty? "<<second.empty()<<endl; // Check if the second string is empty
    cout<<"Is 3rd string empty? "<<third.empty()<<endl; // Check if the third string is empty

    return 0;
}
```

OUTPUT:

```
Execution Failed: Command 'D:\Btech\4th sem\OOPS\ppt_codes\13_string_operations.exe' timed out after 10 secon
```

FILE: 14_access_specifier.cpp

CODE:

```
#include<iostream>
using namespace std;

class Rectangle {
    int width, height;

public:
    void set_values(int ,int);
    int area(){
        return width * height;
    }

};

void Rectangle::set_values(int x, int y){
    width = x;
    height = y;
}

int main(){
    Rectangle rect;
    rect.set_values(5, 10);
    cout << "Area of rectangle: " << rect.area() << endl; // Output will be 50
    return 0;
}
```

OUTPUT:

```
Area of rectangle: 50
```

FILE: 15_access_specifier2.cpp

CODE:

```
#include<iostream>
using namespace std;

class Rectangle {
    int width, height;
public:
    void set_values(int, int);
    int area();
};

void Rectangle::set_values(int x, int y) {
    width = x;
    height = y;
}

int Rectangle::area() {
    return width * height;
}

int main() {
    Rectangle rect,rect2;
    rect.set_values(5, 10);
    cout << "Area of rectangle: " << rect.area() << endl;

    rect2.set_values(7, 12);
    cout << "Area of rectangle 2: " << rect2.area() << endl;
    return 0;
}
```

OUTPUT:

```
Area of rectangle: 50
Area of rectangle 2: 84
```

FILE: 16_constructor.cpp

CODE:

```
#include<iostream>
using namespace std;

class Rectangle {
    int width,height;

    public:
        Rectangle(int,int);
        int area(){
            return width * height;
        }

};

Rectangle::Rectangle(int a ,int b){
    width=a;
    height=b;
}

int main(){
    Rectangle rect(3,4);
    Rectangle rectb(5,6);

    cout<<"rect area: "<<rect.area()<<endl;
    cout<<"rectb area: "<<rectb.area()<<endl;
    return 0;
}
```

OUTPUT:

```
rect area: 12
rectb area: 30
```

FILE: 17_example.cpp

CODE:

```
#include<iostream>
#include<cstring>
using namespace std;


class employee{
    char name[80]; //private by default

    public:
        //these are public
        void putname(const char *n);
        void getname(char *n) const;

    private:
        //now private again
        double wage;

    public:
        //back to public
        void putwage(double w);
        double getwage() const;
};

void employee::putname(const char *n){
    strcpy(name,n);
}

void employee::getname(char *n) const{
    strcpy(n,name);
}

void employee::putwage(double w){
    wage=w;
}

double employee::getwage() const{
    return wage;
}

int main(){
    employee ted;
    char name[80];
    ted.putname("Virat");
    ted.putwage(500);
    ted.getname(name);

    cout<<name<<" makes "<<ted.getwage()<<" crores (INR) per year. ";
    return 0;
}
```

OUTPUT:

```
Virat makes 500 crores (INR) per year.
```

FILE: 18_friend_function.cpp

CODE:

```
#include<iostream>
using namespace std;

class myclass{
    int a,b;
    public:
        friend int sum(myclass x);
        void set_ab(int i,int j);
};

void myclass::set_ab(int i,int j){
    a=i;
    b=j;
}
int sum(myclass x){
    //Bcz sum() is a friend of myclass,it can directly access a and b.
    return x.a +x.b;
}
int main(){
    myclass n;
    n.set_ab(3,4);
    cout<<sum(n);
    return 0;
}
```

OUTPUT:

```
7
```

FILE: 19_friend_class.cpp

CODE:

```
#include<iostream>
using namespace std;

class TwoValues{
    protected:
        int a,b;
    public:
        TwoValues(int i,int j){
            a=i;
            b=j;
        }
        friend class Min;
};

class Min{
    public:
        int min(TwoValues x);
};

int Min::min(TwoValues x){
    return x.a<x.b ? x.a : x.b ;
}

int main(){
    TwoValues ob(10,20);
    Min m;
    cout<<m.min(ob);
    return 0;
}
```

OUTPUT:

```
10
```

FILE: 20_inline_function.cpp

CODE:

```
#include <iostream>
using namespace std;

class myclass{
    int a,b;

    public:
        void init(int i,int j);
        void show();
};

//Create an inline function
inline void myclass::init(int i,int j){
    a=i;
    b=j;
}

//Create another inline function
inline void myclass::show(){
    cout<<a<<" "<<b<<"\n";
}


int main(){
    myclass x;
    x.init(10,20);
    x.show();
    return 0;
}
```

OUTPUT:

```
10 20
```

FILE: 21_default_constructor.cpp

CODE:

```
/*
Default Constructor --> A constructor which has no arguments is
known as default constructor

*/

#include<iostream>
using namespace std;
class Employee{
    public:
    Employee(){
        cout<<"Printed"<<endl;
    }
};

int main(void){
    Employee e1;
    Employee e2;
    return 0;
}
```

OUTPUT:

```
Printed
Printed
```

FILE: 22_static_data_member.cpp

CODE:

```
#include<iostream>
using namespace std;


class shared{
    static int a;
    int b;

    public:
        void set(int i,int j){
            a=i;
            b=j;
        }
        void show();
};

int shared::a; //define a

void shared::show(){
    cout<<"This is a static a: "<<a<<endl;
    cout<<"This is non-static b: "<<b<<endl;
}

int main(){
    shared x,y;

    x.set(1,1); //set a to 1
    x.show();

    y.set(2,2); //change a to 2
    y.show();

    /*
    Here a has been changed for both x and y
    because a is shared by both objects
    */
    x.show();
    return 0;
}
```

OUTPUT:

```
This is a static a: 1
This is non-static b: 1
This is a static a: 2
This is non-static b: 2
This is a static a: 2
This is non-static b: 1
```

FILE: 23_static_data_member_2.cpp

CODE:

```
#include<iostream>
using namespace std;

class shared{
    public:
        static int a;
};

int shared::a;

int main(){
    //initialze a before creating any objects
    shared::a=99;

    cout<<"This is initial value of a: "<<shared::a<<endl;

    shared x;
    cout<<"This is x.a: "<<x.a;
    return 0;
}
```

OUTPUT:

```
This is initial value of a: 99
This is x.a: 99
```

FILE: 24_static_member_function.cpp

CODE:

```
#include<iostream>
using namespace std;

class cl{
    static int resource;
    public:
        static int get_resource();
        void free_resource(){
            resource=0;
        }
};

int cl::resource; //define resource

int cl::get_resource(){
    if(resource){
        return 0; //resource already in use
    }
    else{
        resource=1;
        return 1; //resource allocated to this object
    }
}

int main(){
    cl ob1,ob2;

    //get_resource() is static so may be called independent of any object.
    if(cl::get_resource()){
        cout<<"ob1 has resource"<<endl;
    }
    if(!cl::get_resource()){
        cout<<"ob2 denied resource"<<endl;
    }
    ob1.free_resource();

    //can still call using object syntax
    if(ob2.get_resource()){
        cout<<"ob2 can now use resource"<<endl;
    }

    return 0;
}
```

OUTPUT:

```
ob1 has resource
ob2 denied resource
ob2 can now use resource
```

FILE: 25_destructor.cpp

CODE:

```
#include<iostream>
using namespace std;

class Test{
    public:
        Test(){
            cout<<"Constructor executed"<<endl;
        }

        ~Test(){
            cout<<"Destructor executed "<<endl;
        }
};
int main(){
    Test t;
    return 0;
}
```

OUTPUT:

```
Constructor executed
Destructor executed
```

FILE: 26_destructor_2.cpp

CODE:

```
#include<iostream>
using namespace std;

int i;
class A{
    public:
        ~A(){
            i=10;
        }
};

int foo(){
    i=3;
    A ob;
    return i;
}
int main(){
    cout<<"i= "<<foo()<<endl;
    return 0;
}
```

OUTPUT:

```
i= 3
```

FILE: 27_static_data_member_counter.cpp

CODE:

```
#include<iostream>
using namespace std;

class Counter{
    public:
        static int count;

        Counter(){
            count++;
        }

        ~Counter(){
            count--;
        }
};

// Definition of static member
int Counter::count = 0;

void f(){
    Counter temp;
    cout << "Inside f(): " << Counter::count << endl;
    //temp is destroyed when f() returns
}

int main(){
    Counter o1;
    cout << "Objects in existence: " << Counter::count << endl;

    Counter o2;
    cout << "Objects in existence: " << Counter::count << endl;

    Counter o3;
    cout << "Objects in existence: " << Counter::count << endl;

    f();

    Counter o4;
    cout << "Objects in existence: " << Counter::count << endl;

    return 0;
}
```

OUTPUT:

```
Objects in existence: 1
Objects in existence: 2
Objects in existence: 3
Inside f(): 4
Objects in existence: 4
```

FILE: 28_cons_des.cpp

CODE:

```
//Order of Initialization and Destruction of Global and Local Objects
#include <iostream>
using namespace std;

class myclass{
    public:
        int who;
        myclass(int id);
        ~myclass();

} glob_ob1(1),glob_ob2(2); //Global objects

myclass::myclass(int id){
    cout<<"Initializing "<<id<<"\n";
    who=id;
}

myclass::~myclass(){
    cout<<"Destructing"<<who<<"\n";
}

int main(){
    myclass local_ob1(3);
    cout<<"This will not be first line displayed. \n";

    myclass local_ob2(4);
    return 0;
}
```

OUTPUT:

```
Initializing 1
Initializing 2
Initializing 3
This will not be first line displayed.
Initializing 4
Destructing4
Destructing3
Destructing2
Destructing1
```

FILE: 29_local_classes.cpp

CODE:

```
#include <iostream>
using namespace std;

void f();

int main(){
    f();
    //myclass not known here

    return 0;
}

void f(){
    class myclass{
        int i;
        public:
            void put_i(int n){
                i=n;
            }
            int get_i(){
                return i;
            }
    } ob;

    ob.put_i(10);
    cout<<ob.get_i();
}
```

OUTPUT:

```
10
```

FILE: 30_copy__constructor.cpp

CODE:

```
#include <iostream>
using namespace std;

class Student{
    public:
        int roll;
        string name;
};

int main(){
    Student s1;
    s1.roll=29;
    s1.name="Amit";

    Student s2=s1;
    cout<<"Roll of s2: "<<s2.roll<<endl;
    cout<<"Name of s2: "<<s2.name<<endl;
}
```

OUTPUT:

```
Roll of s2: 29
Name of s2: Amit
```

FILE: 31_copy_constructor.cpp

CODE:

```
#include<iostream>
using namespace std;

class Student{
    int roll;
    string name;

    public:
        Student(int r,string n){
            roll=r;
            name=n;
        }

        //Copy constructor
        Student(const Student &s){
            roll=s.roll;
            name="";
        }
};

int main(){
    Student s1(101,"Amit");
    Student s2=s1;
    return 0;
}
```

OUTPUT:

```
No output printed.
```

FILE: 32_copy_constructor.cpp

CODE:

```
#include <iostream>
using namespace std;

class Point{
    private:
        int x,y;

    public:
        Point(int x1,int y1){
            x=x1;
            y=y1;
        }

        //Copy Constructor
        Point(const Point &p1){
            x=p1.x;
            y=p1.y;
        }

        int getX(){
            return x;
        }

        int getY(){
            return y;
        }
};

int main(){
    Point p1(10,15); //Normal constructor is called here
    Point p2=p1; //Copy constructor is called here

    // Let us access values assigned by constructors
    cout<<"p1.x = "<<p1.getX()<<",p1.y= "<<p1.getY()<<endl;
    cout<<"p2.x = "<<p2.getX()<<",p2.y= "<<p2.getY()<<endl;
}
```

OUTPUT:

```
p1.x = 10,p1.y= 15
p2.x = 10,p2.y= 15
```

FILE: 33_passing_obj_to_function.cpp

CODE:

```
#include<iostream>
using namespace std;

class myclass{
    int i;
    public:
        myclass(int n);
        ~myclass();

        void set_i(int n){
            i=n;
        }

        int get_i(){
            return i;
        }
};

myclass::myclass(int n){
    i=n;
    cout<<"Constructing "<<i<<"\n";
}

myclass::~myclass(){
    cout<<"Destroying "<<i<<endl;
}

void f(myclass obj);

int main(){
    myclass o(1);
    f(o);

    cout<<"This is i in main: ";
    cout<<o.get_i()<<endl;
    return 0;
}

void f(myclass ob){
    ob.set_i(2);
    cout<<"This is local i: "<<ob.get_i()<<endl;
}
```

OUTPUT:

```
Constructing 1
This is local i: 2
Destroying 2
This is i in main: 1
Destroying 1
```

FILE: 34_returning_objects.cpp

CODE:

```
#include<iostream>
using namespace std;

class myclass{
    int i;
    public:
        void set_i(int n){
            i=n;
        }

        int get_i(){
            return i;
        }
};

myclass f();

int main(){
    myclass o;
    o=f();

    cout<<o.get_i()<<endl;
    return 0;
}

myclass f(){
    myclass x;
    x.set_i(1);
    return x;
}
```

OUTPUT:

```
1
```

FILE: 35_obj_assignment.cpp

CODE:

```
#include <iostream>
using namespace std;

class myclass{
    int i;
    public:
        void set_i(int n){
            i=n;
        }

        int get_i(){
            return i;
        }
};

int main(){
    myclass ob1,ob2;
    ob1.set_i(99);

    ob2=ob1;
    cout<<"This is ob2's i: "<<ob2.get_i();

    return 0;
}
```

OUTPUT:

```
This is ob2's i: 99
```

FILE: 36_array_obj.cpp

CODE:

```
#include <iostream>
using namespace std;

class cl{
    int i;
    public:
        void set_i(int j){
            i=j;
        }

        int get_i(){
            return i;
        }
};
int main(){
    cl ob[3];

    int i;

    for(i=0;i<3;i++){
        ob[i].set_i(i+1);
    }

    for(i=0;i<3;i++){
        cout<<ob[i].get_i()<<endl;
    }
    return 0;
}
```

OUTPUT:

```
1
2
3
```

FILE: 37_array_obj.cpp

CODE:

```
#include <iostream>
using namespace std;

class cl{
    int i;
    public:
        //constructor
        cl(int j){
            i=j;
        }

        int get_i(){
            return i;
        }
};
int main(){
    cl ob[3]={1,2,3}; //initializers
    int i;
    for(i=0;i<3;i++){
        cout<<ob[i].get_i()<<endl;
    }

    return 0;
}
```

OUTPUT:

```
1
2
3
```

FILE: 38_array_obj.cpp

CODE:

```
#include <iostream>
using namespace std;

class cl{
    int h,i;
    public:
        cl(int j,int k){
            h=j;
            i=k;
        }
        int get_i(){
            return i;
        }

        int get_h(){
            return h;
        }
};
int main(){
    cl ob[3]={
        cl(1,2),
        cl(3,4),
        cl(5,6)
    };

    int i;
    for(i=0;i<3;i++){
        cout<<ob[i].get_h()<<", "<<ob[i].get_i()<<endl;
    }
    return 0;
}
```

OUTPUT:

```
1, 2
3, 4
5, 6
```

FILE: 39_pointer_to_obj.cpp

CODE:

```
#include <iostream>
using namespace std;

class cl{
    int i;
    public:
        cl(int j){
            i=j;
        }
        int get_i(){
            return i;
        }
};
int main(){
    cl ob(88),*p;
    p=&ob; //get address of ob

    cout<< p->get_i(); //use --> to call get_i()
    return 0;
}
```

OUTPUT:

```
88
```

FILE: 40_pointer_to_obj.cpp

CODE:

```
#include<iostream>
using namespace std;

class cl{
    int i;
    public:
        cl(){
            i=0;
        }
        cl(int j){
            i=j;
        }

        int get_i(){
            return i;
        }
};
int main(){
    cl ob[3]={1,2,3};
    cl *p;

    int i;
    p=ob; //get start of array

    for(i=0;i<3;i++){
        cout<<p->get_i()<<endl;
        p++; //point to next object
    }
    return 0;
}
```

OUTPUT:

```
1
2
3
```

FILE: 41_pointer_to_integer.cpp

CODE:

```
#include <iostream>
using namespace std;

class cl{
    public:
        int i;
        cl(int j){
            i=j;
        }
};

int main(){
    cl ob(1);
    int *p;

    p=&ob.i ;//get address of ob.i
    cout<<*p;

    return 0;
}
```

OUTPUT:

```
1
```

FILE: 42_this_pointer.cpp

CODE:

```
#include <iostream>
using namespace std;

class pwr{
    double b;
    int e;
    double val;

    public:
        pwr(double base,int exp);
        double get_pwr(){
            return val;
        }
};

pwr::pwr(double base,int exp){
    this->b=base;
    this->e=exp;
    this->val=1;

    if(exp==0){
        return;
    }

    for(;exp>0;exp--){
        val=val*b;
    }

}
int main(){
    pwr x(4.0,2), y(2.5,1),z(5.7,0);
    cout<<x.get_pwr()<<" ";
    cout<<y.get_pwr()<<" ";
    cout<<z.get_pwr()<<endl;

    return 0;
}
```

OUTPUT:

```
16 2.5 1
```

FILE: 43_function_overloading.cpp

CODE:

```
#include<iostream>
using namespace std;

int myfunc(int i); //these differ in types of parameters
double myfunc(double i);

int main(){

    cout<<myfunc(10)<<" "; //calls myfunc(int i)
    cout<<myfunc(5.4); //calls myfunc(double i)

    return 0;
}

double myfunc(double i){
    return i;
}

int myfunc(int i){
    return i;
}
```

OUTPUT:

```
10 5.4
```

FILE: 44_function_overloading.cpp

CODE:

```
#include <iostream>

int myfunc(int i);//these differ in no of parameters
int myfunc(int i,int j);

int main(){
    cout<<myfunc(10)<<" "; //calls mufunc(int i)
    cout<<myfunc(4,5); //calls myfunc(int i,int j)
    return 0;
}

int myfunc(int i){
    return i;
}

int myfunc(int i,int j){
    return i*j;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\44_function_overloading.cpp: In function 'int main()':
D:\Btech\4th sem\OOPS\ppt_codes\44_function_overloading.cpp:7:5: error: 'cout' was not declared in this scope
    7 |     cout<<myfunc(10)<<" "; //calls mufunc(int i)
      |     ^~~~
      |     std::cout
In file included from D:\Btech\4th sem\OOPS\ppt_codes\44_function_overloading.cpp:1:
C:/msys64/ucrt64/include/c++/14.2.0/iostream:63:18: note: 'std::cout' declared here
   63 |   extern ostream cout;          ///< Linked to standard output
      |                  ^~~~
```

FILE: 45_overloading_constructor.cpp

CODE:

```
#include <iostream>
using namespace std;

class construct{
    public:
        float area;
        //Constructor with no parameters
        construct(){
            area=0;
        }

        //Constructor with 2 parameters
        construct(int a,int b){
            area=a*b;
        }

        void disp(){
            cout<<area<<endl;
        }
};
int main(){
    //Constructor Overloading with 2 different constructors of class name
    construct o;
    construct o2(10,20);

    o.disp();
    o2.disp();
    return 1;
}
```

OUTPUT:

```
0
200
```

FILE: 46_overloading_constructor.cpp

CODE:

```
#include <iostream>
#include <new>
using namespace std;

class powers{
    int x;
    public:
        //overload constructor 2 ways
        powers(){
            x=0; //no initializer
        }

        powers(int n){
            x=n; //initializer
        }

        int getx(){
            return x;
        }

        void setx(int i){
            x=i;
        }
};

int main(){
    powers ofTwo[]={1,2,4,8,16}; //initialized
    powers ofThree[5]; //uninitialized

    powers *p;
    int i;

    //show powers of two
    cout<<"Powers of two: ";

    for(i=0;i<5;i++){
        cout<<ofTwo[i].getx()<<" ";
    }
    cout<<endl<<endl;

    //Set powers of 3
    ofThree[0].setx(1);
    ofThree[1].setx(3);
    ofThree[2].setx(9);
    ofThree[3].setx(27);
    ofThree[4].setx(81);

    //show powers of three
    cout<<"Powers of three: ";
    for(i=0;i<5;i++){
        cout<<ofThree[i].getx()<<" ";
    }
    cout<<endl<<endl;

    //Dynamically allocate an array
    try{
        p=new powers[5]; //no initialization
    }

    catch(bad_alloc xa){
        cout<<"Allocation failed"<<endl;
        retun 1;
    }

    //initialize dynamic array with powers of two
    for(i=0;i<5;i++){
        p[i].setx(ofTwo[i].getx());
    }

    //show powers of two
    cout<<"Powers of two: ";

    for(i=0;i<5;i++){
        cout<<p[i].getx()<<" ";
    }

    cout<<endl<<endl;

    delete []p;
```

```
    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\46_overloading_constructor.cpp: In function 'int main()':
D:\Btech\4th sem\OOPS\ppt_codes\46_overloading_constructor.cpp:62:9: error: 'retun' was not declared in this
   62 |         retun 1;
      |         ^~~~~
```

FILE: 47_copy_constructor.cpp

CODE:

```
#include <iostream>
#include <string.h>
using namespace std;

class student{
    int rno;
    string name;
    double fee;

    public:
        student(int,string,double);
        student(student &t){ //copy constructor
            rno=t.rno;
            name=t.name;
            fee=t.fee;
        }

        void display();

};

student::student(int no,string n,double f){
    rno=no;
    name=n;
    fee=f;
}

void student::display(){
    cout<<endl<<rno<<"\t"<<name<<"\t"<<fee;
}
int main(){
    student s(1001,"Virat",10000);

    s.display();
    student ram(s); //copy constructor called

    student newOne=s; //copy constructor called

    ram.display();
    newOne.display();
    return 0;
}
```

OUTPUT:

```
1001■Virat■10000
1001■Virat■10000
1001■Virat■10000
```

FILE: 48_function_pointer.cpp

CODE:

```
#include <iostream>
using namespace std;

int multiply(int a;int b){
    return a*b;
}
int main(){
    int (*func)(int,int);
    //func is pointing to the multiply function

    func=multiply;

    int prod=func(15,2);

    cout<<"The value of the product is: "<<prod<<endl;
    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\48_function_pointer.cpp:4:19: error: expected ')' before ';' token
    4 | int multiply(int a;int b){
      |              ~     ^
      |                    )
D:\Btech\4th sem\OOPS\ppt_codes\48_function_pointer.cpp:4:25: error: expected initializer before ')' token
    4 | int multiply(int a;int b){
      |                         ^
D:\Btech\4th sem\OOPS\ppt_codes\48_function_pointer.cpp: In function 'int main()':
D:\Btech\4th sem\OOPS\ppt_codes\48_function_pointer.cpp:11:10: error: invalid conversion from 'int (*)(int)'
   11 |     func=multiply;
      |          ^~~~~~~~
      |          |
      |          int (*)(int)
```

FILE: 49_function_pointer.cpp

CODE:

```
#include <iostream>
using namespace std;

int a=15;
int b=2;

int multiply(){
    return a*b;
}

void print(int (*funcptr)()){
    cout<<"The value of the product is :"<<funcptr<<endl;
}
int main(){
    print(multiply);
    return 0;
}
```

OUTPUT:

```
The value of the product is :1
```

FILE: 50_finding_address_of_overloaded_function.cpp

CODE:

```
#include<iostream>
using namespace std;

int myfunc(int a);
int myfunc(int a,int b);
int main(){
    int (*fp)(int a);
    fp=myfunc; //points to myfunc(int)

    cout<<fp(5);
    return 0;
}

int myfunc(int a){
    return a;
}

int myfunc(int a,int b){
    return a*b;
}
```

OUTPUT:

```
5
```

FILE: 51_operator_function.cpp

CODE:

```
#include<iostream>
using namespace std;

class loc{
    int longitude,latitude;
    public:
        loc(){};
        loc(int lg,int lt){
            longitude=lg;
            latitude=lt;
        }

        void show(){
            cout<<longitude<<" ";
            cout<<latitude<<endl;
        }

        loc operator+(loc op2);
};

//Overload + for loc
loc loc::operator+(loc op2){
    loc temp;
    temp.longitude=op2.longitude + latitude;
    temp.latitude=op2.latitude + latitude;
    return temp;
}
int main(){
    loc ob1(10,20),ob2(5,30);
    ob1.show(); //displays 10 20
    ob2.show(); //displays 5 30

    ob1=ob1+ob2;
    ob1.show(); //displays 15 50
    return 0;
}
```

OUTPUT:

```
10 20
5 30
25 50
```

FILE: 52_operator_overloading.cpp

CODE:

```
#include <iostream>
using namespace std;

class Complex{
    float real;
    float imag;

    public:
        void setNumber(float a,float b){
            this->real=a;
            this->imag=b;
        }

        void show(Complex ob1,Complex ob2);
};

void Complex::show(Complex ob1,Complex ob2){
    cout<<"First complex number: "<<ob1.real<<"+ "<<ob1.imag<<"i"<<endl;
    cout<<"Second complex number: "<<ob2.real<<"+ "<<ob2.imag<<"i"<<endl;
}

int main(){
    Complex a1,a2,a3;

    a1.setNumber(11,12);
    a2.setNumber(3,4);
    a1.show(a1,a2);
    a3=a1+a2;
    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp: In function 'int main()':
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:10: error: no match for 'operator+' (operand t
   28 |     a3=a1+a2;
      |        ~~^~~
      |        |  |
      |        |  Complex
      |        Complex
In file included from C:/msys64/ucrt64/include/c++/14.2.0/string:48,
                 from C:/msys64/ucrt64/include/c++/14.2.0/bits/locale_classes.h:40,
                 from C:/msys64/ucrt64/include/c++/14.2.0/bits/ios_base.h:41,
                 from C:/msys64/ucrt64/include/c++/14.2.0/ios:44,
                 from C:/msys64/ucrt64/include/c++/14.2.0/ostream:40,
                 from C:/msys64/ucrt64/include/c++/14.2.0/iostream:41,
                 from D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:1:
C:/msys64/ucrt64/include/c++/14.2.0/bits/stl_iterator.h:627:5: note: candidate: 'template<class _Iterator> co
  627 |     operator+(typename reverse_iterator<_Iterator>::difference_type __n,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/stl_iterator.h:627:5: note:   template argument deduction/substituti
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/stl_iterator.h:1797:5: note: candidate: 'template<class _Iterator> c
 1797 |     operator+(typename move_iterator<_Iterator>::difference_type __n,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/stl_iterator.h:1797:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
In file included from C:/msys64/ucrt64/include/c++/14.2.0/string:54:
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3598:5: note: candidate: 'template<class _CharT, clas
 3598 |     operator+(const basic_string<_CharT, _Traits, _Alloc>& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3598:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3616:5: note: candidate: 'template<class _CharT, clas
 3616 |     operator+(const _CharT* __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3616:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   mismatched types 'const _CharT*' a
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3635:5: note: candidate: 'template<class _CharT, clas
```

```
 3635 |     operator+(_CharT __lhs, const basic_string<_CharT,_Traits,_Alloc>& __rhs)
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3635:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3652:5: note: candidate: 'template<class _CharT, clas
 3652 |     operator+(const basic_string<_CharT, _Traits, _Alloc>& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3652:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3670:5: note: candidate: 'template<class _CharT, clas
 3670 |     operator+(const basic_string<_CharT, _Traits, _Alloc>& __lhs, _CharT __rhs)
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3670:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3682:5: note: candidate: 'template<class _CharT, clas
 3682 |     operator+(basic_string<_CharT, _Traits, _Alloc>&& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3682:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'std
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3689:5: note: candidate: 'template<class _CharT, clas
 3689 |     operator+(const basic_string<_CharT, _Traits, _Alloc>& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3689:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'con
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3696:5: note: candidate: 'template<class _CharT, clas
 3696 |     operator+(basic_string<_CharT, _Traits, _Alloc>&& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3696:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'std
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3719:5: note: candidate: 'template<class _CharT, clas
 3719 |     operator+(const _CharT* __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3719:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   mismatched types 'const _CharT*' a
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3726:5: note: candidate: 'template<class _CharT, clas
 3726 |     operator+(_CharT __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3726:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'std
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3733:5: note: candidate: 'template<class _CharT, clas
 3733 |     operator+(basic_string<_CharT, _Traits, _Alloc>&& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3733:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'std
   28 |     a3=a1+a2;
      |           ^~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3740:5: note: candidate: 'template<class _CharT, clas
 3740 |     operator+(basic_string<_CharT, _Traits, _Alloc>&& __lhs,
      |     ^~~~~~~~
C:/msys64/ucrt64/include/c++/14.2.0/bits/basic_string.h:3740:5: note:   template argument deduction/substitut
D:\Btech\4th sem\OOPS\ppt_codes\52_operator_overloading.cpp:28:11: note:   'Complex' is not derived from 'std
   28 |     a3=a1+a2;
      |           ^~
```

FILE: 53_imp_points.cpp

CODE:

```
#include <iostream>
using namespace std;

class Complex{
    float real;
    float imag;

    public:
        void setNumber(float a,float b){
            this->real=a;
            this->imag=b;
        }

        void show(Complex ob);
        Complex operator +(Complex c2){
            Complex temp;
            temp.real=real+c2.real;
            temp.imag=imag+c2.imag;
            return temp;
        }
};

void Complex::show(Complex obj){
    cout<<"First complex number: "<<ob.real<<"+"<<ob.imag<<"i"<<endl;
}
int main(){
    Complex a1,a2,a3;
    a1.setNumber(11,12);
    a2.setNumber(3,4);

    a1.show(a1);
    a2.show(a2);

    a3=a1+a2;

    a3.show(a3);

    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\53_imp_points.cpp: In member function 'void Complex::show(Complex)':
D:\Btech\4th sem\OOPS\ppt_codes\53_imp_points.cpp:24:37: error: 'ob' was not declared in this scope; did you
   24 |     cout<<"First complex number: "<<ob.real<<"+"<<ob.imag<<"i"<<endl;
      |                                     ^~
      |                                     obj
```

FILE: 54_operator_overloading_using_friend_function.cpp

CODE:

```
#include <iostream>
using namespace std;

class loc{
    int longitude,latitude;

    public:
        loc(){};
        loc(int lg,int lt){
            longitude=lg;
            latitude=lt;
        }

        void show(){
            cout<<longitude<<" ";
            cout<<latitude<<endl;
        }

        friend loc operator +(loc op1,loc op2);
};

//Now, + is overloaded using friend function
loc operator +(loc op1,loc op2){
    loc temp;
    temp.longitude=op1.longitude + op2.longitude;
    temp.latitude=op1.latitude + op2.latitude;

    return temp;
}
int main(){
    loc ob1(10,20), ob2(5,30);
    ob1=ob1 + ob2;

    ob1.show();
    return 0;
}
```

OUTPUT:

```
15 50
```

FILE: 55_overloading.cpp

CODE:

```
#include<iostream>
using namespace std;

class loc{
    int longitude,latitude;
    public:
        loc(){} //needed to construct
        loc(int lg,int lt){
            longitude=lg;
            latitude=lt;
        }

        void show(){
            cout<<longitude<<" ";
            cout<<latitude<<endl;
        }

        loc operator+(loc op2);
        loc operator=(loc op2);
        loc operator++();
};

loc loc::operator+(loc op2){
    loc temp;
    temp.longitude=op2.longitude + longitude;
    temp.latitude=op2.latitude + latitude;

    return temp;
}

//Overload assignment for loc
loc loc::operator=(loc op2){
    longitude=op2.longitude;
    latitude=op2.latitude;
    return *this; // i.e, return object that generated call
}

//Overload prefix ++ for loc
loc loc::operator++(){
    longitude++;
    latitude++;
    return *this;
}

int main(){
    cout<<"Initializing objects ..."<<endl;
    loc ob1(10,20),ob2(5,30),ob3(90,90);
    ob1.show();
    ob2.show();
    ++ob1;

    cout<<"After incrementing ob1...."<<endl;
    ob1.show(); //displays 11 21
    ob2=++ob1;

    cout<<"Status of ob1 and ob2"<<endl;
    ob1.show(); //displays 12 22
    ob2.show(); //displays 12 22

    return 0;
}
```

OUTPUT:

```
Initializing objects ...
10 20
5 30
After incrementing ob1....
11 21
Status of ob1 and ob2
12 22
12 22
```

FILE: 56_string_comparision.cpp

CODE:

```
#include<iostream>
using namespace std;
class CompareString{
    public:
        char str[25];
        //Parameterized Constructor
        CompareString(char str1[]){
            //Initialize the string to class objects
            strcpy(this->str,str1);
        }

        //Overloading '=='
        int operator==(CompareString s2){
            if(strcmp(str,s2.str)==0){
                return 1;
            }
            else{
                retun 0;
            }
        }

        //Overloading '<='
        int operator<=(CompareString s3){
            if(strlen(str)<=strlen(s3.str)){
                return 1;
            }
            else{
                return 0;
            }
        }

        //Overloading '>=' under a function
        int operator>=(CompareString s3){
            if(strlen(str)>=strlen(s3.str)){
                return 1;
            }

            else{
                return 0;
            }
        }
};

void compare(CompareString s1,CompareString s2){
    if(s1==s2){
        cout<<s1.str<<" is equal to "<<s2.str<<endl;
    }
    else{
        cout<<s1.str<<" is not equal to "<<s2.str<<endl;

        if(s1>=s2){
            cout<<s1.str<<" is greater than "<<s2.str<<endl;
        }
        else{
            cout<<s2.str<<" is greater than "<<s1.str<<endl;
        }
    }
}

void testcase1(){
    //Declaring 2 strings
    char str1[]="Rupam";
    char str2[]="For IT Dept";

    //Declaring and initializing the class with above 2 strings
    CompareString s1(str1);
    CompareString s2(str2);

    cout<<"Comparing \""<<s1.str<<"\" and \""<<s2.str<<"\""<<endl;

    compare(s1,s2);
}

void testcase2(){
    //Declaring 2 strings
    char str1[]="Virat";
    char str2[]="Virat";

    //Declaring and initializing the class with above 2 strings
```

```
    CompareString s1(str1);
    CompareString s2(str2);

    cout<<"\n\n Comparing \""<< s1.str<<"\" and \""<<s2.str<<"\""<<endl;

    compare(s1,s2);
}

//Driver code
int main(){
    testcase1();
    testcase2();

    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp: In constructor 'CompareString::CompareString(char*
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:9:13: error: 'strcpy' was not declared in this scop
    9 |            strcpy(this->str,str1);
      |            ^~~~~
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:2:1: note: 'strcpy' is defined in header '<cstring>
    1 | #include<iostream>
  +++ |+#include <cstring>
    2 | using namespace std;
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp: In member function 'int CompareString::operator==(
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:14:16: error: 'strcmp' was not declared in this sco
   14 |            if(strcmp(str,s2.str)==0){
      |               ^~~~~
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:14:16: note: 'strcmp' is defined in header '<cstrin
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:18:17: error: 'retun' was not declared in this scop
   18 |                retun 0;
      |                ^~~~
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp: In member function 'int CompareString::operator<=(
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:24:16: error: 'strlen' was not declared in this sco
   24 |            if(strlen(str)<=strlen(s3.str)){
      |               ^~~~~
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:24:16: note: 'strlen' is defined in header '<cstrin
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp: In member function 'int CompareString::operator>=(
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:34:16: error: 'strlen' was not declared in this sco
   34 |            if(strlen(str)>=strlen(s3.str)){
      |               ^~~~~
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:34:16: note: 'strlen' is defined in header '<cstrin
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp: In member function 'int CompareString::operator==(
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:20:9: warning: control reaches end of non-void func
   20 |         }
      |         ^
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp: In member function 'int CompareString::operator>=(
D:\Btech\4th sem\OOPS\ppt_codes\56_string_comparision.cpp:41:9: warning: control reaches end of non-void func
   41 |         }
      |         ^
```

FILE: 57_new_operator.cpp

CODE:

```
#include<iostream>
using namespace std;

int main(){
    int *ptr1=NULL;
    ptr1=new int;

    float *ptr2=new float(299.121);
    int *ptr3=new int[28];

    *ptr1=28;

    cout<<"Value of pointer variable 1: "<<*ptr1<<endl;
    cout<<"Value of pointer variable 2: "<<*ptr2<<endl;

    if(!ptr3){
        cout<<"Allocation of memory failed"<<endl;
    }

    else{
        for(int i=10;i<15;i++){
            ptr3[i]=i+1;
        }
        cout<<"Value of store in block of memory: ";
        for(int i=10;i<15;i++){
            cout<<ptr3[i]<<" ";
        }
    }

    delete ptr1;
    delete ptr2;

    delete[] ptr3;

    return 0;
}
```

OUTPUT:

```
Value of pointer variable 1: 28
Value of pointer variable 2: 299.121
Value of store in block of memory: 11 12 13 14 15
```

FILE: 58_overloading_new_delete_operator.cpp

CODE:

```
#include<iostream>
#include<stdlib.h>

using namespace std;
class student{
    string name;
    int age;

    public:
        student(){
            cout<<"Constructor is called"<<endl;
        }

        student(string name,int age){
            this->name=name;
            this->age=age;
        }

        void display(){
            cout<<"Name: "<<name<<endl;
            cout<<"Age: "<<age<<endl;
        }

        void *operator new(size_t size){
            cout<<"Overloading new operator with size: "<<size<<endl;
            void *p= ::operator new(size);

            //void *p=malloc(size); will also work fine
            return p;
        }

        void operator delete(void *p){
            cout<<"Overloading delete operator"<<endl;
            free(p);
        }
};

int main(){
    student *p=new student("Yash",24);
    p->display();
    delete p;
}
```

OUTPUT:

```
Overloading new operator with size: 40
Name: Yash
Age: 24
Overloading delete operator
```

FILE: 59_inheritance.cpp

CODE:

```
#include<iostream>
using namespace std;

class base{
    int i,j;
    public:
        void set(int a,int b){
            i=a;
            j=b;
        }

        void show(){
            cout<<i<<" "<<j<<endl;
        }
};

class derived:public base{
    int k;
    public:
        derived(int x){
            k=x;
        }

        void showk(){
            cout<<k<<endl;
        }
};

int main(){
    derived ob(3);
    ob.set(1,2); //access member of base
    ob.show(); //access member of base
    ob.showk(); //uses member of derived class

    return 0;
}
```

OUTPUT:

```
1 2
3
```

FILE: 60_inheritance.cpp

CODE:

```
#include <iostream>
using namespace std;

class base{
    int i,j;

    public:
        void set(int a,int b){
            i=a;
            j=b;
        }

        void show(){
            cout<<i<<" "<<j<<endl;
        }
};

//Public elements of base are private in derived
class derived:private base{
    int k;
    public:
        derived(int x){
            k=x;
        }

        void showk(){
            cout<<k<<endl;
        }
};

int main(){
    derived ob(3);
    ob.set(1,2); //error,can't access set()
    ob.show(); // error, can't access show()
    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp: In function 'int main()':
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp:33:11: error: 'void base::set(int, int)' is inaccessible w
   33 |     ob.set(1,2); //error,can't access set()
      |     ~~~~~~^~~~
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp:8:14: note: declared here
    8 |         void set(int a,int b){
      |              ^~~
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp:33:11: error: 'base' is not an accessible base of 'derived
   33 |     ob.set(1,2); //error,can't access set()
      |     ~~~~~~^~~~
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp:34:12: error: 'void base::show()' is inaccessible within t
   34 |     ob.show(); // error, can't access show()
      |     ~~~~~~~^~
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp:13:14: note: declared here
   13 |         void show(){
      |              ^~~~
D:\Btech\4th sem\OOPS\ppt_codes\60_inheritance.cpp:34:12: error: 'base' is not an accessible base of 'derived
   34 |     ob.show(); // error, can't access show()
      |     ~~~~~~~^~
```

FILE: 61_public_inheritance.cpp

CODE:

```
#include <iostream>
using namespace std;

class Base{
    private:
        int pvt=1;

    protected:
        int prot=2;

    public:
        int pub=3;

    // function to access private member
    int getPVT(){
        return pvt;
    }
};

class PublicDerived:public Base{
    public:
        //function to access protected member from Base

        int getProt(){
            return prot;
        }
};

int main(){
    PublicDerived object1;
    cout<<"Private = "<<object1.getPVT()<<endl;
    cout<<"Protected = "<<object1.getProt()<<endl;
    cout<<"Public = "<<object1.pub<<endl;

    return 0;
}
```

OUTPUT:

```
Private = 1
Protected = 2
Public = 3
```

FILE: 62_protected_inheritance.cpp

CODE:

```
#include <iostream>
using namespace std;

class Base{
    private:
        int pvt=1;

    protected:
        int prot=2;

    public:
        int pub=3;

        //function to access private member
        int getPVT(){
            return pvt;
        }
};

class ProtectedDerived:protected Base{
    public:
        //function to access protected member from Base

        int getProt(){
            return prot;
        }

        //function to access public member from Base
        int getPub(){
            return pub;

        }
};

int main(){
    ProtectedDerived object1;
    cout<<"Private cannot be accessed."<<endl;
    cout<<"Protected = "<<object1.getProt()<<endl;
    cout<<"Public = "<<object1.getPub()<<endl;

    return 0;
}
```

OUTPUT:

```
Private cannot be accessed.
Protected = 2
Public = 3
```

FILE: 63_private_inheritance.cpp

CODE:

```
#include<iostream>
using namespace std;

class Base{
    private:
        int pvt=1;

    protected:
        int prot=2;

    public:
        int pub=3;

        //function to access private member
        int getPVT(){
            return pvt;
        }
};

class PrivateDerived : private Base{
    public:
        //function to access protected member from Base

        int getProt(){
            return prot;
        }

        // function to access private member
        int getPub(){
            return pub;
        }
};

int main(){
    PrivateDerived object1;
    cout<<"Private cannot be accessed."<<endl;
    cout<<"Protected = "<<object1.getProt()<<endl;
    cout<<"Public = "<<object1.getPub()<<endl;

    return 0;
}
```

OUTPUT:

```
Private cannot be accessed.
Protected = 2
Public = 3
```

FILE: 64_inheriting_multiple_base_classes.cpp

CODE:

```
#include <iostream>
using namespace std;

class base1{
    protected:
        int x;

    public:
        void showx(){
            cout<<x<<endl;
        }
};

class base2{
    protected:
        int y;

    public:
        void showy(){
            cout<<y<<endl;
        }
};

//Inherit multiple base classes
class derived: public base1,public base2{
    public:
        void set(int i,int j){
            x=i;
            y=j;
        }
};

int main(){
    derived ob;
    ob.set(10,20); //provided by derived
    ob.showx(); //from base1
    ob.showy(); //from base2

    return 0;
}
```

OUTPUT:

```
10
20
```

FILE: 65_constructor_destructor_inheritance.cpp

CODE:

```
#include <iostream>
using namespace std;

class base{
    public:
        base(){
            cout<<"Constructing base "<<endl;
        }

        ~base(){
            cout<<"Destructing base "<<endl;
        }
};

class derived:public base{
    public:
        derived(){
            cout<<"Constructing derived"<<endl;
        }

        ~derived(){
            cout<<"Destructing derived "<<endl;
        }
};

int main(){
    derived ob;
    //do nothing but construct and destruct ob
    return 0;
}
```

OUTPUT:

```
Constructing base
Constructing derived
Destructing derived
Destructing base
```

FILE: 66_inheritance.cpp

CODE:

```
#include <iostream>
using namespace std;

class base{
    public:
    base(){
        cout<<"Constructing base "<<endl;
    }

    ~base(){
        cout<<"Destructing base "<<endl;
    }
};

class derived1 : public base{
    public:
        derived1(){
            cout<<"Constructing derived1"<<endl;
        }

        ~derived1(){
            cout<<"Destructing derived1"<<endl;
        }
};

class derived2 : public derived1{
    public:
        derived2(){
            cout<<"Constructing derived2"<<endl;
        }

        ~derived2(){
            cout<<"Destructing derived2"<<endl;
        }
};

int main(){
    derived2 ob;

    //construct and destruct ob
    return 0;
}
```

OUTPUT:

```
Constructing base
Constructing derived1
Constructing derived2
Destructing derived2
Destructing derived1
Destructing base
```

FILE: 67_multiple_base_classes.cpp

CODE:

```
#include <iostream>
using namespace std;

class base1{
    public:
        base1(){
            cout<<"Constructing base1"<<endl;
        }

        ~base1(){
            cout<<"Destructing base1"<<endl;
        }
};

class base2{
    public:
        base2(){
            cout<<"Constructing base2"<<endl;
        }

        ~base2(){
            cout<<"Destructing base2"<<endl;
        }
};

class derived: public base1,public base2{
    public:
        derived(){
            cout<<"Constructing derived"<<endl;
        }

        ~derived(){
            cout<<"Destructing derived"<<endl;
        }
};

int main(){
    derived ob;
    // construct and destruct ob
    return 0;
}
```

OUTPUT:

```
Constructing base1
Constructing base2
Constructing derived
Destructing derived
Destructing base2
Destructing base1
```

FILE: 68_passing_parameters_to_base_class_constructor.cpp

CODE:

```
#include <iostream>
using namespace std;

class base{
    protected:
        int i;
        public:
            base(int x){
                i=x;
                cout<<"Constructing base"<<endl;
            }

            ~base(){
                cout<<"Destructing base"<<endl;
            }
};

class derived: public base{
    int j;
    public:
        //derived uses x; y is passed along to base.
        derived(int x,int y):base(y){
            j=x;
            cout<<"Constructing derived"<<endl;
        }

        ~(derived){
            cout<<"Destructing derived"<<endl;
        }

        void show(){
            cout<<i<<" "<<j<<endl;
        }
};

int main(){
    derived ob(3,4);
    ob.show(); //displays 4 3

    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\68_passing_parameters_to_base_class_constructor.cpp:27:10: error: expected cl
   27 |         ~(derived){
      |          ^
```

FILE: 69.passing_parameters_to_base_class_constructor.cpp

CODE:

```
#include<iostream>
using namespace std;

class base1{
    protected:
        int i;
        public:
            base1(int x){
                i=x;
                cout<<"Constructing base1"<<endl;
            }

            ~base1(){
                cout<<"Destructing base1"<<endl;
            }
};

class base2{
    protected:
        int k;
        public:
            base2(int x){
                k=x;
                cout<<"Constructing base2"<<endl;
            }

            ~base2(){
                cout<<"Destructing base1"<<endl;
            }
};

class derived: public base1, public base2{
    int j;
    public:
        derived(int x,int y,int z) : base1(y), base2(z){
            j=x;
            cout<<"Constructing derived"<<endl;
        }

        ~derived(){
            cout<<"Destructing derived"<<endl;
        }

        void show(){
            cout<<i<<" "<<j<<" "<<k<<endl;
        }
};

int main(){
    derived ob(3,4,5);
    ob.show(); //displays 4 3 5
    return 0;
}
```

OUTPUT:

```
Constructing base1
Constructing base2
Constructing derived
4 3 5
Destructing derived
Destructing base1
Destructing base1
```

FILE: 70_virtual_base_classes.cpp

CODE:

```
//This program contains an error and will not compile
```

OUTPUT:

```
COMPILATION ERROR:
C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/14.2.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:/msys6
C:/M/B/src/mingw-w64/mingw-w64-crt/crt/crtexewin.c:67:(.text.startup+0xc5): undefined reference to `WinMain'
collect2.exe: error: ld returned 1 exit status
```

FILE: new.cpp

CODE:

```
#include <iostream>
using namespace std;

class A {
protected:
    int age = 10;
};

class B : public A {
public:
    void show() {
        printf("%d", age);
    }
};

class C:public B{
    public:
        void show(){
            printf("%d",age);
        }
}

int main() {
    B obj;
    obj.show();

    return 0;
}
```

OUTPUT:

```
COMPILATION ERROR:
D:\Btech\4th sem\OOPS\ppt_codes\new.cpp:21:2: error: expected ';' after class definition
   21 | }
      |  ^
      |  ;
```

FILE: new_1.cpp

CODE:

```
#include <iostream>
using namespace std;

class A {
protected:
    int age = 10;
};

class B : protected A {
public:
    int a;

    void show() {
        a = age;
        printf("%d\n", a);
    }
};

class C : protected B {
public:
    void display() {
        show();
    }
};

int main() {
    B obj;
    obj.show();

    printf("%d\n", obj.a);

    return 0;
}
```

OUTPUT:

```
10
10
```